

מחברת 6: ויזואליזציה ומפות

מבוא למדעי הרוח הדיגיטליים | אוניברסיטת אריאל | תשפ"ו

פרופ' שי גורדין

מה נלמד במחברת זו? ויזואליזציה = המרת נתונים לייצוג ויזואלי שמקל על הבנה וגילוי דפוסים.

נושאים:

1. עקרונות ויזואליזציה טובה
2. גרפים סטטיים עם Matplotlib ו-Seaborn
3. גרפים אינטראקטיביים עם Plotly
4. מפות גיאוגרפיות עם Folium
5. GIS בארכיאולוגיה ומדעי הרוח
6. ציר זמן (Timeline)

🎯 **הקשר לפרויקט:** נמפה אתרים ארכיאולוגיים וניצור ויזואליזציות של הנתונים שנאספו לאורך הקורס.

חלק א: עקרונות ויזואליזציה טובה

מדוע ויזואליזציה חשובה?

"A picture is worth a thousand words" – אבל רק אם הוא מדויק ומכבד את הנתונים.

עקרונות Edward Tufte:

עיקרון	תיאור	דוגמה
Data-Ink Ratio	מקסמו "דיו של נתונים" – הסירו קישוטים	הסירו gridlines מיותרות
Chartjunk	הימנעו מקישוטים ללא תוכן	ללא 3D, ללא הצללות
Lie Factor	ייצוג יחסי נאמן לנתונים	ציר Y שמתחיל ב-0
Small Multiples	השוואות ע"י ריבוי גרפים זהים	פאנל גרפים לפי שנה

סוגי גרפים וייעודם:

גרף	מתי להשתמש
עמודות (Bar)	השוואת קטגוריות
קו (Line)	מגמות לאורך זמן

גרף	מתי להשתמש
פיזור (Scatter)	קשר בין שני משתנים
היסטוגרמה	התפלגות ערכים
חום (Heatmap)	מטריצות ומתאמים
מפה (Map)	נתונים גיאוגרפיים

📖 לקריאה: Tufte, E. (1983). *The Visual Display of Quantitative Information*. Graphics Press.

🚀 לפני שמתחילים – מדריך הגדרה מהירה

שלב 1: פתחו את המחברת ב-Google Colab

לחצו על הכפתור הכחול "Open in Colab" בדף הקורס.

שלב 2: שמרו עותק אישי

File → Save a copy in Drive
(חובה – אחרת השינויים שלכם לא יישמרו!)

שלב 3: הריצו את התאים לפי הסדר – מלמעלה למטה

- **Shift + Enter** = מריץ תא נוכחי ועובר לבא אחריו
- **Ctrl + Enter** = מריץ תא נוכחי ונשאר בו

⌚ זמן משוער

- שיעור: ~70 דקות
- 💡 המחברת הזו עוסקת בהמחשה חזותית – כדאי לפתוח Colab בחלון גדול!

💡 מה נלמד היום

- גרפים סטטיסטיים עם Matplotlib ו-Seaborn
- ויזואליזציה אינטראקטיבית עם Plotly
- מפות גאוגרפיות עם Folium

? נתקעתם?

1. קראו את הודעת השגיאה
2. שאלו את Gemini ב-AI Studio – העתיקו את השגיאה ושאלו
3. שאלו את המרצה / הקבוצה

```
In [ ]: # התקנת ספריות ויזואליזציה
!pip install plotly folium seaborn -q
print("✅ ספריות ויזואליזציה הותקנו")
print("• plotly - גרפים אינטראקטיביים")
print("• folium - מפות גיאוגרפיות")
print("• seaborn - סטטיסטיים מתקדמים")
```

```
In [ ]: # יבוא ספריות
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import folium
from IPython.display import display, HTML, IFrame
import json
import os

matplotlib.rcParams['axes.unicode_minus'] = False
sns.set_style("whitegrid")

print("✅ ספריות יובאו")

# — נתוני אתרים ארכיאולוגיים לדוגמה
# נתונים מייצגים אתרים ארכיאולוגיים בישראל
SITES_DATA = [
    {"שם": "מגידו", "lat": 32.584, "lon": 35.183, "צפון": "אזור", "דרום": "צפון", "מרכז": "צפון", "7": 10},
    {"שם": "חצור", "lat": 33.017, "lon": 35.567, "צפון": "אזור", "דרום": "צפון", "מרכז": "צפון", "7": 9},
    {"שם": "לכיש", "lat": 31.561, "lon": 34.848, "צפון": "אזור", "דרום": "דרום", "מרכז": "דרום", "7": 9},
    {"שם": "ירושלים", "lat": 31.769, "lon": 35.216, "צפון": "אזור", "דרום": "מרכז", "מרכז": "מרכז", "7": 10},
    {"שם": "ערד", "lat": 31.261, "lon": 35.118, "צפון": "אזור", "דרום": "דרום", "מרכז": "דרום", "7": 8},
    {"שם": "בית שאן", "lat": 32.498, "lon": 35.500, "צפון": "אזור", "דרום": "צפון", "מרכז": "צפון", "7": 8},
    {"שם": "מגרשות", "lat": 31.801, "lon": 34.748, "צפון": "אזור", "דרום": "מרכז", "מרכז": "מרכז", "7": 7},
    {"שם": "כרמל (מערות)", "lat": 32.732, "lon": 35.061, "צפון": "אזור", "דרום": "צפון", "מרכז": "צפון", "7": 9},
    {"שם": "ים המלח", "lat": 31.500, "lon": 35.467, "צפון": "אזור", "דרום": "מרכז", "מרכז": "מרכז", "7": 9},
    {"שם": "תמנע", "lat": 29.776, "lon": 35.001, "צפון": "אזור", "דרום": "דרום", "מרכז": "דרום", "7": 8},
    {"שם": "קיסריה", "lat": 32.495, "lon": 34.889, "צפון": "אזור", "דרום": "מרכז", "מרכז": "מרכז", "7": 8},
    {"שם": "בית גוברין", "lat": 31.609, "lon": 34.894, "צפון": "אזור", "דרום": "דרום", "מרכז": "דרום", "7": 7}
]

df_sites = pd.DataFrame(SITES_DATA)
print(f"✅ {len(df_sites)} אתרים ארכיאולוגיים נטענו")
display(df_sites.head(5))
```

חלק ב: גרפים סטטיים – Matplotlib ו-Seaborn

מתי גרפים סטטיים?

• פרסום – מאמרים, כנסים

```
In [ ]: # =====
# גרפים סטטיים – ניתוח אתרים ארכיאולוגיים
# =====

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('ניתוח אתרים ארכיאולוגיים בישראל', fontsize=16, fontweight='bold')

REGION_COLORS = {'צפון': '#e74c3c', '#2980b9': 'מרכז', '#27ae60': 'דרום'}

# — גרף 1: שכבות לפי אתר —
ax1 = axes[0, 0]
df_sorted = df_sites.sort_values('שכבות', ascending=True)
colors1 = [REGION_COLORS[r] for r in df_sorted['אזור']]
bars = ax1.barh(df_sorted['שם'], df_sorted['שכבות'], color=colors1, alpha=0.8)
ax1.set_xlabel('מספר שכבות', fontsize=11)
ax1.set_title('שכבות התיישבות לפי אתר', fontsize=12, fontweight='bold')
for bar, val in zip(bars, df_sorted['שכבות']):
    ax1.text(bar.get_width() + 0.2, bar.get_y() + bar.get_height()/2,
             str(val), va='center', fontsize=9)

# — גרף 2: התפלגות אזורית —
ax2 = axes[0, 1]
region_counts = df_sites['אזור'].value_counts()
wedge_colors = [REGION_COLORS[r] for r in region_counts.index]
wedges, texts, autotexts = ax2.pie(
    region_counts, labels=region_counts.index,
    colors=wedge_colors, autopct='%1.0f%%',
    startangle=90, textprops={'fontsize': 11}
)
ax2.set_title('חלוקה אזורית', fontsize=12, fontweight='bold')

# — גרף 3: שכבות לפי חשיבות —
ax3 = axes[1, 0]
scatter_colors = [REGION_COLORS[r] for r in df_sites['אזור']]
sc = ax3.scatter(df_sites['חשיבות'], df_sites['שכבות'],
                 c=scatter_colors, s=df_sites['חשיבות'] * 20,
                 alpha=0.8, edgecolors='white', linewidth=1.5)
for _, row in df_sites.iterrows():
    ax3.annotate(row['שם'], (row['חשיבות'], row['שכבות']),
                 textcoords='offset points', xytext=(5, 3), fontsize=8, color='black')
ax3.set_xlabel('חשיבות היסטורית', fontsize=11)
ax3.set_ylabel('מספר שכבות', fontsize=11)
ax3.set_title('שכבות לעומת חשיבות', fontsize=12, fontweight='bold')

# — גרף 4: boxplot תקופות —
ax4 = axes[1, 1]
region_data = [df_sites[df_sites['אזור'] == r]['שכבות'].tolist()
                for r in ['דרום', 'מרכז', 'צפון']]
bp = ax4.boxplot(region_data, labels=['דרום', 'מרכז', 'צפון'],
                 patch_artist=True)
for patch, color in zip(bp['boxes'], ['#e74c3c', '#2980b9', '#27ae60']):
    patch.set_facecolor(color)
```

```

    patch.set_alpha(0.7)
ax4.set_ylabel('מספר שכבות', fontsize=11)
ax4.set_title('התפלגות שכבות לפי אזור', fontsize=12, fontweight='bold')

# מקרא
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor=c, label=r) for r, c in REGION_COLORS.items()]
fig.legend(handles=legend_elements, loc='lower center', ncol=3,
           fontsize=10, title='אזור', bbox_to_anchor=(0.5, -0.02))

plt.tight_layout()
plt.savefig('01_static_charts.png', dpi=150, bbox_inches='tight')
plt.show()
print("🖨️ 01: הגרף נשמר static_charts.png")

```

חלק ג: גרפים אינטראקטיביים עם Plotly

יתרונות Plotly:

- **Hover:** מידע מפורט בעת ריחוף
- **Zoom:** זום ידני
- **Click:** הדגשה/הסתרה
- **Export:** PNG, HTML, SVG

Plotly Express לעומת Plotly Graph Objects

- **px** (Express) = ממשק פשוט, קוד קצר
- **go** (Graph Objects) = שליטה מלאה, קוד ארוך יותר

```

In [ ]: # =====
# Plotly - גרפים אינטראקטיביים
# =====

print("📊 יוצר גרפים אינטראקטיביים")

# — גרף פיזור אינטראקטיבי —
fig_scatter = px.scatter(
    df_sites,
    x='חשיבות',
    y='שכבות',
    size='שכבות',
    color='אזור',
    hover_name='שם',
    hover_data={'תקופה': True, 'שכבות': True, 'חשיבות': True},
    color_discrete_map={'צפון': '#e74c3c', '#2980b9': 'מרכז', '#27ae60': 'דרום'},
    title='אתרים ארכיאולוגיים - שכבות לעומת חשיבות',
    labels={'חשיבות': 'חשיבות היסטורית', 'שכבות': 'מספר שכבות'},
    size_max=40
)
fig_scatter.update_layout(
    font_family="Arial",

```

```

        title_font_size=16,
        legend_title_text='אזור',
        height=450
    )
fig_scatter.write_html('02_scatter_interactive.html')
fig_scatter.show()
print("✅ 02 : נשמר_02_scatter_interactive.html")

# — גרף עמודות מקובצות —
fig_bar = px.bar(
    df_sites,
    x='שם',
    y='שכבות',
    color='אזור',
    color_discrete_map={'צפון': '#e74c3c', '#2980b9': 'מרכז', '#27ae60': 'דרום'},
    title='מספר שכבות לכל אתר',
    labels={'שם': 'שם האתר', 'שכבות': 'מספר שכבות'},
    hover_data=['תקופה', 'חשיבות']
)
fig_bar.update_layout(xaxis_tickangle=-45, height=450)
fig_bar.write_html('03_bar_interactive.html')
fig_bar.show()
print("✅ 03 : נשמר_03_bar_interactive.html")

```

חלק ד: מפות גיאוגרפיות עם Folium

מהו Folium?

Folium = ספריית Python ליצירת מפות אינטראקטיביות המבוססות על **Leaflet.js**.

סוגי מפות:

סוג	שימוש
Marker	נקודות בודדות (אתרים, ממצאים)
CircleMarker	עיגולים בגדלים שונים (כמות)
Heatmap	צפיפות נקודות
Choropleth	צביעת אזורים לפי ערך
GeoJSON	שכבות ישויות גיאוגרפיות

GIS בארכיאולוגיה:

GIS (Geographic Information System) = מערכת מידע גיאוגרפי.

- ניתוח מרחבי של ממצאים
- זיהוי דפוסי התיישבות
- שחזור נופי עבר (Landscape Archaeology)
- ניתוח חזות (Viewshed Analysis)

```
In [ ]: # =====
# Folium - מפות גיאוגרפיות
# =====

print("🗺️ יוצר מפות גיאוגרפיות")

# — מפה 1: אתרים ארכיאולוגיים
m1 = folium.Map(
    location=[31.5, 35.0], # מרכז ישראל
    zoom_start=8,
    tiles='OpenStreetMap'
)

# הוספת כותרת
title_html = '''
<div style="position: fixed; top: 10px; left: 50%; transform: translateX(-50%);
    background: white; padding: 8px 16px; border-radius: 8px;
    border: 2px solid #2980b9; font-size: 16px; font-weight: bold; z-index: 1000;">
    אתרים ארכיאולוגיים בישראל
</div>
'''

m1.get_root().html.add_child(folium.Element(title_html))
```

```

REGION_COLORS_FOLIUM = {'צפון': 'red', 'מרכז': 'blue', 'דרום': 'green'}

for _, site in df_sites.iterrows():
    # גודל העיגול = חשיבות
    site_name = site['שם']
    site_period = site['תקופה']
    site_layers = site['שכבות']
    site_importance = site['חשיבות']
    site_region = site['אזור']

    folium.CircleMarker(
        location=[site['lat'], site['lon']],
        radius=site['חשיבות'] * 2,
        color=REGION_COLORS_FOLIUM.get(site['אזור'], 'gray'),
        fill=True,
        fill_opacity=0.7,
        popup=folium.Popup(
            '<div dir="rtl" style="text-align:right; font-family:Arial;">'
            '<b>' + site_name + '</b><br>'
            '<b>תקופה:</b>' + site_period + '<br>'
            '<b>שכבות:</b>' + str(site_layers) + '<br>'
            '<b>חשיבות:</b>' + str(site_importance) + '/10<br>'
            '<b>אזור:</b>' + site_region +
            '</div>',
            max_width=200
        ),
        tooltip=site_name
    ).add_to(m1)

# מקרא
legend_html = '''
<div style="position: fixed; bottom: 30px; right: 10px;
        background: white; padding: 10px; border-radius: 8px;
        border: 2px solid gray; font-family: Arial; font-size: 13px; z-i
        <b>מקרא - אזורים</b><br>
        צפון &nbsp;   מרכז &nbsp;   דרום<br>
        <i>גודל עיגול = חשיבות</i>
    </div>
    '''

m1.get_root().html.add_child(folium.Element(legend_html))

m1.save('04_archaeology_map.html')
print("✅ 04: נשמרה מפה archaeology_map.html")

# Colab-הצגה ב
display(IFrame('04_archaeology_map.html', width='100%', height=550))

```

```

In [ ]: # =====
# מפת צפיפות (Heatmap) – Folium
# =====

try:
    from folium.plugins import HeatMap

    # 2 מפה: Heatmap של אתרים לפי חשיבות

```



```

m2 = folium.Map(location=[31.5, 35.0], zoom_start=8, tiles='CartoDB posi

# נתונים: [lat, lon, weight]
heat_data = [[row['lat'], row['lon'], row['חשיבות']] for _, row in df_si

HeatMap(
    heat_data,
    min_opacity=0.3,
    max_zoom=18,
    radius=30,
    blur=20,
    gradient={0.4: 'blue', 0.65: 'lime', 1: 'red'})
).add_to(m2)

m2.save('05_heatmap.html')
print("✅ Heatmap 05 נשמר: heatmap.html")
display(IFrame('05_heatmap.html', width='100%', height=500))

except ImportError:
    print("⚠️ folium.plugins לא זמין - מדלג על folium.plugins")
    print("    pip install folium[plugins]")

```

חלק ה: ציר זמן (Timeline)

ציר זמן במדעי הרוח:

ציר זמן = ויזואליזציה של אירועים/תקופות על ציר כרונולוגי.

שימושים:

- ציר זמן של ממלכות – מהכנעני עד הביזנטי
- ציר זמן של חפירות – מי חפר מתי ומה מצא
- ציר זמן של שכבות – שכבות חפירה = ציר זמן אנכי
- ציר זמן של פרסומים – תולדות מחקר שדה

```

In [ ]: # =====
# Plotly - ציר זמן היסטורי
# =====

# נתוני תקופות היסטוריות בארץ ישראל
PERIODS_DATA = [
    {"#9": "צבע", "קטגוריה": "קדם-היסטוריה", "סיום": 3500, "התחלה": 100000},
    {"#8": "צבע", "קטגוריה": "פרהיסטוריה", "סיום": 3300, "התחלה": 4500},
    {"#7": "צבע", "קטגוריה": "ברונזה", "סיום": 2000, "התחלה": 3300},
    {"#6": "צבע", "קטגוריה": "ברונזה", "סיום": 1550, "התחלה": 2000},
    {"#5": "צבע", "קטגוריה": "ברונזה", "סיום": 1200, "התחלה": 1550},
    {"#4": "צבע", "קטגוריה": "ברזל", "סיום": 1000, "התחלה": 1200},
    {"#3": "צבע", "קטגוריה": "ברזל", "סיום": 586, "התחלה": 1000},
    {"#2": "צבע", "קטגוריה": "קלאסי", "סיום": 332, "התחלה": 586},
    {"#1": "צבע", "קטגוריה": "קלאסי", "סיום": 37, "התחלה": 332},
    {"#0": "צבע", "קטגוריה": "קלאסי", "סיום": 324, "התחלה": 37},
    {"#16": "צבע", "קטגוריה": "ביזנטי", "סיום": 638, "התחלה": 324}
]

```

```

{ "צבע" : "צבע", "מימה" : "מימה", "קטגוריה" : "קטגוריה", "סיום" : "סיום", "התחלה" : "התחלה", "1099" : "1099", "638" : "638" },
{ "צבע" : "צבע", "מימה" : "מימה", "קטגוריה" : "קטגוריה", "סיום" : "סיום", "התחלה" : "התחלה", "1291" : "1291", "1099" : "1099" },
{ "#7" : "צבע", "מאוזר" : "מאוזר", "קטגוריה" : "קטגוריה", "סיום" : "סיום", "התחלה" : "התחלה", "1517" : "1517", "1291" : "1291" },
{ "#95" : "צבע", "מאוזר" : "מאוזר", "קטגוריה" : "קטגוריה", "סיום" : "סיום", "התחלה" : "התחלה", "1917" : "1917", "1517" : "1517" },
{ "#2" : "צבע", "מודרני" : "מודרני", "קטגוריה" : "קטגוריה", "סיום" : "סיום", "התחלה" : "התחלה", "1948" : "1948", "1917" : "1917" },
{ "#34495" : "צבע", "מודרני" : "מודרני", "קטגוריה" : "קטגוריה", "סיום" : "סיום", "התחלה" : "התחלה", "2025" : "2025", "1948" : "1948" }
]

df_periods = pd.DataFrame(PERIODS_DATA)

# — Plotly ויזואליזציה עם —
fig_timeline = go.Figure()

for i, row in df_periods.iterrows():
    duration = row['סיום'] - row['התחלה']
    fig_timeline.add_trace(go.Bar(
        name=row['תקופה'],
        x=[duration],
        y=[row['קטגוריה']],
        orientation='h',
        base=[row['התחלה']],
        marker_color=row['צבע'],
        marker_line_width=1,
        marker_line_color='white',
        hovertemplate=(
            "<b>" + row['תקופה'] + "</b><br>"
            "התחלה: " + str(abs(row['התחלה'])) + (" לפנה"ס" if row['התחלה'] < 0 else " לפנה"ס" if row['סיום'] < 0 else " לספירה הנוצרית"),
            "סיום: " + str(abs(row['סיום'])) + (" לפנה"ס" if row['סיום'] < 0 else " לספירה הנוצרית"),
            "משך: " + str(abs(duration)) + " שנים<extra></extra>"
        ),
        showlegend=False
    ))

# קו הספירה הנוצרית
fig_timeline.add_vline(x=0, line_dash="dash", line_color="gray", line_width=1,
    annotation_text="0 (ספירה נוצרית)", annotation_pos="bottom")

fig_timeline.update_layout(
    title="<b>ציר זמן - תקופות היסטוריות בארץ ישראל</b>",
    xaxis_title="שנה (שלילי = לפנה"ס)",
    yaxis_title="קטגוריה",
    barmode='stack',
    height=500,
    showlegend=False,
    font_family="Arial",
    title_font_size=16,
    plot_bgcolor='#f8f9fa',
)

fig_timeline.write_html('06_timeline.html')
fig_timeline.show()
print("✅ 06 ציר הזמן נשמר: 06_timeline.html")

```

```

In [ ]: # =====
# Heatmap מתאמים – Seaborn
# =====

```

```

print("Heatmap יוצר")

# נתוני ויקיפדיה (מדומים - מחברת 1)
wiki_data = {
    'קטגוריה': ['ארכיאולוגיה', 'ארכיאולוגיה', 'ארכיאולוגיה', 'היסטוריה', 'היסטוריה', 'היסטוריה', 'DH', 'DH', 'DH'],
    'ערך': ['מגידו', 'לכיש', 'ירושלים (ארכ)', 'ממלכת ישראל', 'ממלכת יהודה', 'תקופת בית שני', 'OCR', 'NLP', 'GIS'],
    'קישורים_נכנסים': [18, 23, 12, 78, 54, 67, 89, 32, 45],
    'קישורים_יוצאים': [22, 19, 15, 83, 48, 72, 95, 28, 38],
    'KB_אורך_ערך': [85, 62, 180, 145, 98, 165, 35, 67, 48],
}

df_wiki = pd.DataFrame(wiki_data)

# מטריצת מתאמים
numeric_cols = ['KB_קישורים_נכנסים', 'KB_קישורים_יוצאים', 'אורך_ערך']
corr_matrix = df_wiki[numeric_cols].corr()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# — Heatmap מתאמים —
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt='.2f',
    cmap='RdYlGn',
    center=0,
    square=True,
    linewidths=0.5,
    ax=axes[0],
    cbar_kws={'label': 'מתאם'}
)
axes[0].set_title('מטריצת מתאמים - מדדי ויקיפדיה', fontsize=13, fontweight='bold')

# — Heatmap: ערכים לפי קטגוריה —
pivot = df_wiki.pivot_table(
    values='קישורים_נכנסים',
    index='קטגוריה',
    aggfunc='mean'
)

# יצירת מטריצה לדוגמה
matrix_data = {
    'קישורים_נכנסים_ממוצע': [17.67, 66.33, 55.33],
    'אורך_ממוצע': [50, 136, 109],
    'קישורים_יוצאים_ממוצע': [18.67, 67.67, 53.67]
}

df_heatmap = pd.DataFrame(matrix_data, index=['היסטוריה', 'ארכיאולוגיה', 'DH'])

sns.heatmap(
    df_heatmap,
    annot=True,
    fmt='.1f',
    cmap='Blues',

```

```

        linewidths=0.5,
        ax=axes[1]
    )
    axes[1].set_title('מדדים ממוצעים לפי קטגוריה', fontsize=13, fontweight='bold')
    axes[1].set_xlabel('')

    plt.tight_layout()
    plt.savefig('07_heatmaps.png', dpi=150, bbox_inches='tight')
    plt.show()
    print("📁 Heatmaps 07 : נשמרו _heatmaps.png")

```

```

In [ ]: # =====
# שמירת כל הנתונים וסיכום
# =====

print("📁 שומר נתונים...")

df_sites.to_csv('sites_data.csv', index=False, encoding='utf-8-sig')
print("✅ sites_data.csv")

df_periods.to_csv('periods_timeline.csv', index=False, encoding='utf-8-sig')
print("✅ periods_timeline.csv")

print("\n📁 קבצי ויזואליזציה שנוצרו:")
files_created = [
    ('01_static_charts.png', 'גרפים סטטיים - ניתוח אתרים'),
    ('02_scatter_interactive.html', 'פיזור אינטראקטיבי - Plotly'),
    ('03_bar_interactive.html', 'עמודות אינטראקטיביות - Plotly'),
    ('04_archaeology_map.html', 'מפת אתרים ארכיאולוגיים - Folium'),
    ('05_heatmap.html', 'מפת צפיפות - Folium'),
    ('06_timeline.html', 'ציר זמן היסטורי - Plotly'),
    ('07_heatmaps.png', 'מתאמים - Seaborn'),
]

for fname, desc in files_created:
    if os.path.exists(fname):
        size = os.path.getsize(fname)
        print(f"📁 {fname:<38} {size:>8,} bytes ← {desc}")

print("\n📁 מחברת 6 הושלמה!")
print()
print("🎓 כל 6 המחברות הושלמו!")
print()
print("📖 הקורס המלא:")
print("01 - ויקיפדיה ורשתות ידע")
print("02 - קריאה מרחוק")
print("03 - OCR לעברית")
print("04 - NLP נושאים")
print("05 - AI למדעי הרוח")
print("06 - ויזואליזציה ומפות ← אתם כאן!")

```

תרגילים

תרגיל 1 - בסיסי

הוסיפו 5 אתרים ארכיאולוגיים נוספים לנתונים והציגו אותם על המפה.
מצאו את הקואורדינטות ב-Google Maps או ב-Wikidata.

תרגיל 2 – בינוני

צרו מפה כרונולוגית: הוסיפו ל-Folium שכבות (LayerControl) לפי תקופה.
כל תקופה = שכבה נפרדת שניתן להדליק/לכבות.

תרגיל 3 – מתקדם

חברו את ויזואליזציית הרשת ממחברת 1 עם ויזואליזציית המפה:

- צמתים ברשת = אתרים ארכיאולוגיים
- לחיצה על צומת → פתיחת מפה עם מיקום האתר

משאבים

- /Folium Documentation: <https://python-visualization.github.io/folium>
- /Plotly Documentation: <https://plotly.com/python>
- ESRI ArcGIS Online – GIS מקצועי
- QGIS – GIS קוד פתוח
- Tufte, E. (1983). *The Visual Display of Quantitative Information*. Graphics Press.
- Cairo, A. (2016). *The Truthful Art*. New Riders
- Spatial Humanities

סיכום הקורס

כל הנושאים שלמדנו:

1. ויקיפדיה כמקור נתונים + רשתות ידע
2. קריאה מרחוק + ניתוח טקסט
3. OCR לטקסטים עבריים
4. NLP + Topic Modeling
5. AI למדעי הרוח
6. ויזואליזציה + מפות

הפרויקט הסופי: שילוב של כל הכלים לחקר נושא ארכיאולוגי/היסטורי/DH שבחרתם. בהצלחה!

מה להגיש:

1. ויזואליזציה מקורית – צרו גרף אחד (סוג לבחירתכם) לנתונים שאספתם במחברות קודמות (רשת ויקיפדיה / קורפוס / נושאים)
2. כותרת ותיאור – הוסיפו כותרת, תוויות לצירים ואגדה ברורה לגרף
3. פרשנות – בפסקה קצרה: מה הגרף מראה? מה ניתן ללמוד ממנו?

כיצד להגיש:

1. **File → Save a copy in Drive** – ודאו שהמחברת שמורה
2. **File → Download → Download .ipynb** – הורידו את הקובץ
3. העלו ל-Moodle

עצה: אם נתקעתם – שאלו את Gemini!

פתחו טאב חדש ב-[Google AI Studio](#) ושאלו:

`Python (matplotlib / folium)` – אני לומד/ת ויזואליזציה ב
"נתקלתי בשגיאה הזו: "[הדביקו את השגיאה כאן]
?מה המשמעות ואיך אפשר לפתור